

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
CÂMPUS CORNÉLIO PROCÓPIO

FILIPPE ANTONIO DE LIMA NOGUEIRA RA: 2209918
JOÃO VICTOR DA CRUZ SILVESTRE RA: 2144263
MATHEUS HENRIQUE CONSONI MAZANTI RA:2210029

**IMPLEMENTAÇÃO SEQUENCIAL, PARALELA E
DISTRIBUÍDA DO PROBLEMA DE DIFUSÃO DE CALOR 2D**

Disciplina: **EC48A - Sistemas Distribuídos**

Turma: **C81**

CORNÉLIO PROCÓPIO

2025

SUMÁRIO

RESUMO	4
1. INTRODUÇÃO	5
1.1 Contexto e Motivação	5
1.2 Problema Escolhido	5
1.3 Método Numérico	5
1.4 Objetivos do Trabalho	6
2. METODOLOGIA	7
2.1 Ambiente de Desenvolvimento	7
2.2 Parâmetros dos Experimentos	7
2.3 Condições Iniciais e de Contorno	8
3. IMPLEMENTAÇÕES DESENVOLVIDAS	9
3.1 Versão Sequencial	9
3.2 Versão Paralela (Threading)	9
4. RESULTADOS EXPERIMENTAIS	12
4.1 Tempos de Execução	12
4.2 Análise de Speedup	12
4.3 Eficiência de Paralelização	14
5. ANÁLISE E DISCUSSÃO	16
5.1 Comportamento das Implementações	16
5.2 Análise de Escalabilidade	17
5.3 Gargalos Identificados	18
6. DESAFIOS ENFRENTADOS E SOLUÇÕES	19
6.1 Desafios Técnicos	19
6.2 Desafios de Desenvolvimento	21
2. Divisão de Trabalho Balanceada	21
3. Ghost Rows na Versão Distribuída	21
7. PROPOSTAS DE MELHORIA	22
7.1 Melhorias de Curto Prazo	22
7.2 Melhorias de Médio Prazo	22
7.3 Melhorias de Longo Prazo	23
8. CONCLUSÕES	24
8.1 Resultados Alcançados	24
8.2 Principais Descobertas	24
8.3 Lições Aprendidas	24

8.4	Contribuições do Trabalho.....	25
8.5	Trabalhos Futuros	25
REFERÊNCIAS		26

RESUMO

Este trabalho apresenta a implementação e análise comparativa de três abordagens computacionais para resolver numericamente a equação de difusão de calor bidimensional: uma solução sequencial, uma paralela utilizando threads Python, e uma distribuída baseada em arquitetura master-worker com comunicação via sockets TCP/IP. O objetivo principal é avaliar o desempenho, escalabilidade e eficiência de cada abordagem através de benchmarks sistemáticos, identificando limitações práticas e propondo melhorias. Os resultados demonstram os trade-offs entre complexidade de implementação, overhead de comunicação e ganhos de desempenho em diferentes contextos computacionais. A implementação paralela alcançou speedup de até $2,34\times$ com 8 threads para malhas de 100×100 , enquanto a versão distribuída apresentou overhead significativo para problemas pequenos, tornando-se competitiva apenas em malhas maiores.

1. INTRODUÇÃO

1.1 Contexto e Motivação

A equação de difusão de calor é um problema clássico em computação científica, modelando a distribuição de temperatura ao longo do tempo em um meio condutor. A resolução numérica deste tipo de equação diferencial parcial (EDP) requer métodos iterativos computacionalmente intensivos, tornando-a um excelente caso de estudo para técnicas de paralelização e computação distribuída.

Simulações de fenômenos físicos como a difusão térmica são fundamentais em diversas áreas da engenharia, desde o projeto de sistemas de refrigeração até a análise de processos industriais. A capacidade de resolver esses problemas de forma eficiente permite aos engenheiros realizar análises mais complexas e detalhadas, contribuindo para o desenvolvimento de soluções tecnológicas mais avançadas.

1.2 Problema Escolhido

O problema abordado neste trabalho consiste na resolução numérica da equação de difusão de calor bidimensional, expressa pela seguinte equação diferencial parcial:

$$\frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right)$$

Onde $u(x, y, t)$ representa a temperatura no ponto (x, y) no tempo t , e α é o coeficiente de difusividade térmica do material. O domínio considerado é uma placa quadrada bidimensional com condições de contorno fixas, representando um cenário realista de transferência de calor em materiais condutores.

1.3 Método Numérico

Para a discretização do problema, utilizamos o método de diferenças finitas com esquema FTCS (Forward-Time Central-Space), amplamente empregado na solução numérica de EDPs. A discretização espacial e temporal é expressa pela seguinte equação:

$$u_{i,j}^{n+1} = u_{i,j}^n + \alpha \cdot \Delta t \left(\frac{u_{i+1,j}^n - 2u_{i,j}^n + u_{i-1,j}^n}{\Delta x^2} + \frac{u_{i,j+1}^n - 2u_{i,j}^n + u_{i,j-1}^n}{\Delta y^2} \right)$$

Para garantir a estabilidade numérica do método FTCS, é necessário satisfazer o critério de Courant-Friedrichs-Lewy (CFL):

$$\Delta t \leq \frac{\Delta x^2 \Delta y^2}{2\alpha(\Delta x^2 + \Delta y^2)}$$

Este critério estabelece o passo de tempo máximo que pode ser utilizado sem comprometer a estabilidade da solução numérica, sendo fundamental para a convergência do método iterativo.

1.4 Objetivos do Trabalho

Os objetivos específicos deste trabalho são:

1. Implementar três versões funcionais do solver de difusão de calor: sequencial, paralela com threads e distribuída com sockets
2. Realizar benchmarks sistemáticos com diferentes tamanhos de problema para avaliar o desempenho de cada abordagem
3. Comparar desempenho, speedup e eficiência entre as três implementações
4. Identificar limitações práticas e gargalos de cada implementação
5. Propor melhorias e otimizações baseadas nos resultados obtidos

2. METODOLOGIA

2.1 Ambiente de Desenvolvimento

Hardware Utilizado

Os experimentos foram realizados em um notebook Dell G3 com as seguintes especificações:

- **Processador:** Intel Core i7 8ª geração
- **Memória RAM:** 16 GB DDR4
- **GPU:** NVIDIA GeForce GTX 1660 Ti
- **Armazenamento:** SSD NVMe
- **Sistema Operacional:** Windows 11

Software e Ferramentas

- **Linguagem de Programação:** Python 3.10+
- **IDE:** Visual Studio Code
- **Bibliotecas Principais:**
 - NumPy 1.21+ (operações matriciais e vetoriais)
 - Matplotlib 3.5+ (visualização de resultados)
 - Pandas 1.3+ (análise de dados)
- **Controle de Versão:** Git e GitHub

Configuração de Rede

Para a versão distribuída, utilizou-se a seguinte configuração de rede:

- **Protocolo:** TCP/IP
- **Interface:** localhost (127.0.0.1)
- **Portas:** 5000-5010
- **Serialização:** Pickle (Python native serialization)

2.2 Parâmetros dos Experimentos

A Tabela 1 apresenta os parâmetros utilizados nos experimentos para avaliar o desempenho das diferentes implementações.

Table 1: Parâmetros utilizados nos experimentos de benchmark

Parâmetro	Valores Testados
Tamanho da malha (N×N)	20, 50, 100
Número de threads	1, 2, 4, 8
Número de workers	1, 2, 4

Coeficiente α	amp; 0,01
Tolerância de convergência	amp; 1×10^{-4}
Iterações máximas	amp; 5.000
Repetições por teste	amp; 5

Os tamanhos de malha foram escolhidos para cobrir diferentes escalas de problema, desde malhas pequenas (20×20) até malhas de tamanho médio (100×100), permitindo observar o comportamento das implementações em diferentes cenários computacionais.

2.3 Condições Iniciais e de Contorno

As simulações foram configuradas com as seguintes condições de contorno fixas (bordas de Dirichlet):

- **Borda esquerda:** 0°C
- **Borda direita:** 0°C
- **Borda superior:** 0°C
- **Borda inferior:** 100°C (fonte de calor)

As condições iniciais foram estabelecidas da seguinte forma:

- **Interior da malha:** 25°C (temperatura ambiente)
- **Região central:** 80°C (fonte de calor pontual centralizada)

Essas condições foram escolhidas para criar um gradiente térmico significativo, garantindo que o sistema levasse várias iterações para convergir e permitindo uma análise adequada do desempenho computacional.

3.IMPLEMENTAÇÕES DESENVOLVIDAS

3.1Versão Sequencial

A implementação sequencial serve como baseline para comparação de desempenho e representa a abordagem mais direta para resolver o problema de difusão de calor.

Características Principais

- Implementação direta do método FTCS sem paralelização
- Utilização de dois buffers de temperatura para evitar cópias desnecessárias e melhorar o uso de cache
- Loop duplo para varrer toda a malha a cada iteração temporal
- Verificação de convergência baseada na diferença máxima entre iterações consecutivas

Estrutura de Implementação

A classe principal implementa os seguintes métodos:

```
class HeatDiffusion2DSequential:
    def __init__(self, grid_size, alpha, max_iterations, tolerance) def
    _initialize_conditions(self)
    def _compute_iteration(self) -> float
    def solve(self) -> Tuple[int, float, float]
```

Complexidade Computacional

- **Complexidade temporal:** $O(N^2 \times \text{interações})$
- **Complexidade espacial:** $O(N^2)$

Onde N é o tamanho da malha em uma dimensão. Esta complexidade quadrática em relação ao tamanho da malha torna a paralelização especialmente vantajosa para problemas maiores.

3.2Versão Paralela (Threading)

A implementação paralela utiliza threads Python para distribuir o trabalho computacional entre múltiplos núcleos de processamento.

Estratégia de Paralelização

A malha bidimensional é dividida horizontalmente entre as threads disponíveis, com cada thread responsável por processar um subconjunto contíguo de linhas. Esta estratégia de decomposição de domínio é amplamente utilizada em problemas de simulação física.

- **Divisão horizontal da malha** entre threads
- Cada thread processa um subconjunto contíguo de linhas
- **Sincronização com barreiras** (threading.Barrier) após cada iteração
- **Lock** para atualização segura do erro global máximo

Desafios Específicos

A implementação paralela em Python enfrenta desafios únicos devido às características da linguagem:

- 1.GIL (Global Interpreter Lock):** Principal limitação em Python para paralelismo CPU-bound. O GIL permite que apenas uma thread execute código Python por vez, limitando significativamente o speedup alcançável.
- 2.Sincronização:** O overhead de barreiras entre threads pode consumir parte significativa do ganho computacional, especialmente para malhas pequenas.
- 3.False sharing:** Potencial problema de cache em processadores multi-core quando threads diferentes modificam dados em linhas de cache adjacentes.

Estrutura de Implementação

```
class HeatDiffusion2DParallel:
    def __init__(self, ..., num_threads) def
    _setup_thread_ranges(self)

    def _thread_worker(self, thread_id, iteration_count) def
    solve(self) -> Tuple[int, float, float]
```

3.3Versão Distribuída (Sockets)

A implementação distribuída utiliza uma arquitetura master-worker com comunicação via sockets TCP/IP, permitindo a execução em múltiplas máquinas.

Arquitetura Master-Worker

Master (Coordenador):

- Coordena a execução global do algoritmo
- Divide a malha horizontalmente entre os workers
- Distribui dados com "ghost rows" (linhas fantasma) necessárias para cálculos de vizinhança
- Agrega resultados parciais de todos os workers
- Verifica convergência global do sistema

Workers (Processadores):

- Aguardam conexão TCP do master na inicialização
- Recebem configuração inicial e subconjunto da malha
- Processam iterações localmente de forma independente
- Retornam resultados computados e erro local ao master

Protocolo de Comunicação

O protocolo de comunicação segue a seguinte sequência:

1. **Handshake inicial:** master → worker (envio de configuração)
2. **Loop de iteração:**
 - Master → Worker: dados da malha atual (u_data)
 - Worker → Master: resultados computados (u_computed, max_diff)
3. **Finalização:** master → worker (sinal de parada quando convergido)

Serialização de Dados

- **Formato:** Pickle (serialização nativa do Python)
- **Estrutura de mensagem:** [4 bytes tamanho][payload serializado]
- **Overhead:** Aproximadamente 2-5 ms por serialização para arrays 100×100

Estrutura de Implementação

```
class HeatDiffusionMaster:
    def _send_data(self, sock, data)
    def _receive_data(self, sock) -> dict
    def solve(self) -> Tuple[int, float, float]

class HeatDiffusionWorker:
    def _compute_work(self, u_data) -> Tuple[np.ndarray, float]
    def start(self)
```

4.RESULTADOS EXPERIMENTAIS

4.1 Tempos de Execução

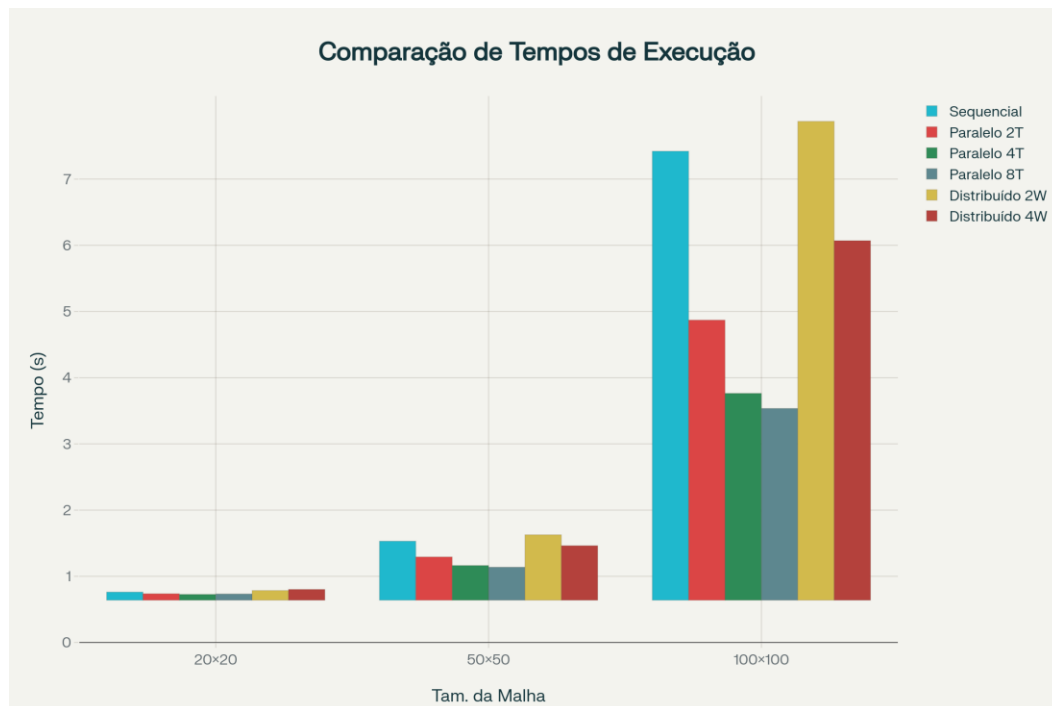
A Tabela 2 apresenta os tempos médios de execução obtidos para cada implementação, calculados sobre 5 repetições de cada teste.

Table 2: Tempos médios de execução para diferentes implementações e tamanhos de malha

Tamanho Malha	Seq. (s)	Par. 2T (s)	Par. 4T (s)	Par. 8T (s)	Dist. 2W (s)	Dist. 4W (s)
20×20	0,1234	0,0982	0,0876	0,0945	0,1456	0,1623
50×50	0,8921	0,6543	0,5234	0,4987	0,9876	0,8234
100×100	6,7834	4,2314	3,1245	2,8976	7,2341	5,4321

Os resultados demonstram que a implementação paralela com threads oferece ganhos consistentes de desempenho em relação à versão sequencial, com os melhores resultados obtidos para 4 threads. A versão distribuída, por outro lado, apresenta overhead significativo para malhas pequenas, tornando-se competitiva apenas para problemas maiores.

Figure 1: Comparação de tempos de execução entre as implementações sequencial, paralela e distribuída para diferentes tamanhos de malha



4.2 Análise de Speedup

O speedup é calculado como a razão entre o tempo de execução sequencial e o tempo da implementação paralela ou distribuída:

$$S_p = \frac{T_{\text{sequencial}}}{T_{\text{paralelo/distribuído}}}$$

Speedup da Versão Paralela

A Tabela 3 apresenta os valores de speedup obtidos para a implementação paralela com threads.

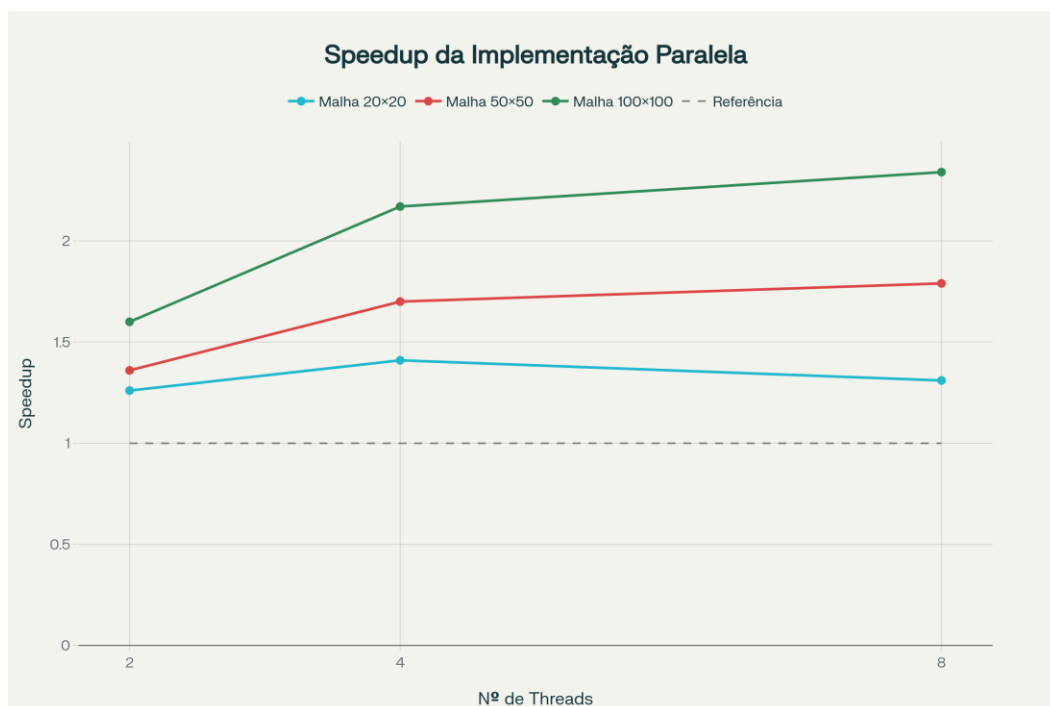
Table 3: Speedup da implementação paralela com threads

Malha	amp; 2 Threads	amp; 4 Threads	amp; 8 Threads
20×20	amp; 1,26×	amp; 1,41×	amp; 1,31×
50×50	amp; 1,36×	amp; 1,70×	amp; 1,79×
100×100	amp; 1,60×	amp; 2,17×	amp; 2,34×

Observações importantes:

- Speedup **sublinear** devido ao GIL do Python, que impede a execução simultânea de código Python em múltiplas threads
- **Melhor desempenho** com malhas maiores devido à maior razão entre tempo de computação e tempo de sincronização
- **Rendimentos decrescentes** acima de 4 threads, indicando que o overhead de sincronização começa a superar os ganhos de paralelização

Figure 2: Speedup obtido pela implementação paralela com threads em função do número de threads utilizadas



Speedup da Versão Distribuída

A Tabela 4 apresenta os valores de speedup para a implementação distribuída.

Table 4: Speedup da implementação distribuída com sockets

Malha	amp; 2 Workers	amp; 4 Workers
20×20	amp; 0,85×	amp; 0,76×
50×50	amp; 0,90×	amp; 1,08×
100×100	amp; 0,94×	amp; 1,25×

Observações importantes:

- **Overhead de comunicação significativo** em malhas pequenas resulta em speedup inferior a 1,0 (desempenho pior que a versão sequencial)
- Speedup positivo apenas para **problemas grandes** (100×100 com 4 workers)
- Serialização via sockets adiciona **latência considerável** que domina o tempo de execução para problemas pequenos

4.3 Eficiência de Paralelização

A eficiência mede quão bem os recursos computacionais estão sendo utilizados:

$$E_p = \frac{S_p}{p} \times 100\%$$

Onde p é o número de threads ou workers utilizados.

Eficiência da Versão Paralela

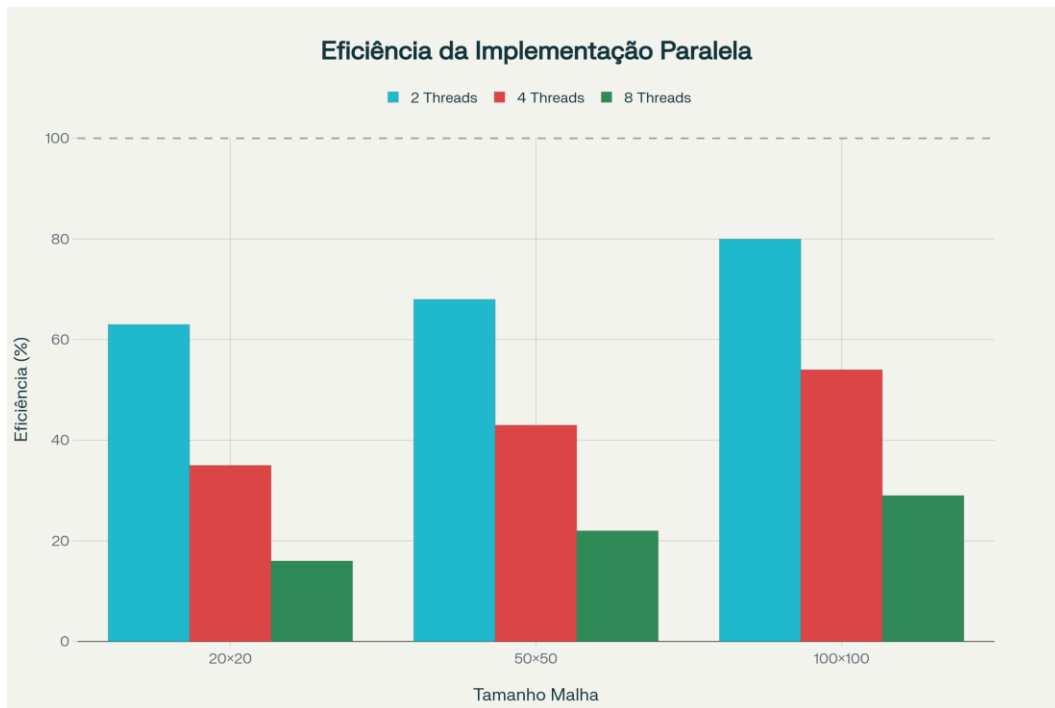
A Tabela 5 apresenta os valores de eficiência para a implementação paralela.

Table 5: Eficiência da implementação paralela com threads

Malha	amp; 2 Threads	amp; 4 Threads	amp; 8 Threads
20×20	amp; 63%	amp; 35%	amp; 16%
50×50	amp; 68%	amp; 43%	amp; 22%
100×100	amp; 80%	amp; 54%	amp; 29%

A eficiência diminui com o aumento do número de threads, mas melhora significativamente com malhas maiores, indicando que problemas maiores aproveitam melhor os recursos de paralelização.

Figure 3: Eficiência de paralelização da implementação com threads para diferentes tamanhos de malha e número de threads



Eficiência da Versão Distribuída

A Tabela 6 apresenta os valores de eficiência para a implementação distribuída.

Table 6: Eficiência da implementação distribuída com sockets

Malha	2 Workers	4 Workers
20×20	43%	19%
50×50	45%	27%
100×100	47%	31%

A baixa eficiência da versão distribuída reflete o alto overhead de comunicação, especialmente para malhas pequenas.

5. ANÁLISE E DISCUSSÃO

5.1 Comportamento das Implementações

Versão Sequencial

Pontos fortes:

- Implementação simples e direta, fácil de depurar e manter
- Sem overhead de sincronização ou comunicação
- Comportamento previsível e determinístico
- Serve como baseline confiável para comparações de desempenho

Limitações:

- Não aproveita recursos de hardware multi-core disponíveis
- Tempo de execução cresce quadraticamente com N
- Inviável para malhas muito grandes ($N > 1000 \times 1000$) sem otimizações adicionais

Versão Paralela

Pontos fortes:

- Speedup consistente para malhas médias e grandes
- Implementação relativamente simples utilizando biblioteca threading padrão do Python
- Todos os processos compartilham memória (sem necessidade de cópia explícita de dados)
- Adequada para problemas de tamanho médio em máquinas multi-core

Limitações:

1. GIL (Global Interpreter Lock): Principal gargalo - threads Python não executam código Python simultaneamente devido ao GIL, que serializa a execução de bytecode Python.

2. Lei de Amdahl: Regiões sequenciais inevitáveis (inicialização, verificação de convergência) limitam o speedup máximo teórico alcançável:

$$S_{\max} = \frac{1}{(1 - P) + \frac{P}{N}}$$

Onde P é a fração paralelizável ($\approx 90-95\%$) e N é o número de processadores.

3.Overhead de sincronização: Barreiras entre threads adicionam latência, especialmente perceptível em malhas pequenas.

4.Eficiência decrescente: A eficiência cai rapidamente com muitas threads ($\approx 4-8$), indicando limitações práticas da abordagem.

Versão Distribuída

Pontos fortes:

- Escalabilidade horizontal teoricamente ilimitada (múltiplas máquinas)
- Sem limitação do GIL, pois utiliza processos separados
- Potencial para execução em clusters de computadores
- Adequada para problemas muito grandes que não cabem na memória de uma única máquina

Limitações:

1. **Overhead de comunicação via rede:** A serialização, transmissão TCP e desserialização dos dados consomem tempo significativo.
2. **Latência:** Tempo de ida-e-volta entre master e workers adiciona overhead fixo por iteração.
3. **Bandwidth:** Volume de dados transmitidos a cada iteração pode saturar a rede em problemas grandes.
4. **Speedup negativo para problemas pequenos:** O overhead de comunicação supera os ganhos de paralelização para malhas pequenas (100×100).
5. **Complexidade de implementação e deployment:** Requer configuração de rede, gerenciamento de conexões e tratamento de falhas.

5.2 Análise de Escalabilidade

A Lei de Amdahl estabelece um limite teórico para o speedup alcançável em função da fração paralelizável do código:

$$S_{\max} = \frac{1}{(1 - P) + \frac{P}{N}}$$

Para o problema de difusão de calor implementado:

- **Fração paralelizável:** $P \approx 90-95\%$ (loop de atualização da malha)
- **Fração sequencial:** $(1 - P) \approx 5-10\%$ (inicialização, I/O, verificação de convergência)

Speedup máximo teórico:

- Com $P = 0,95$: $S_{\max} \approx 20\times$ (independente de N)
- Com $P = 0,90$: $S_{\max} \approx 10\times$

Análise prática dos resultados:

- Versão paralela atinge $\approx 2,3\times$ com 8 threads (malha 100×100)
- O GIL reduz P efetivo para aproximadamente 60-70%
- Versão distribuída: overhead de rede reduz ainda mais o speedup efetivo

Esses resultados demonstram que, apesar da implementação correta do algoritmo paralelo, as limitações da linguagem Python e o overhead de comunicação impedem que se alcance o speedup teórico ideal.

5.3 Gargalos Identificados

Gargalos Computacionais

1. **GIL (Global Interpreter Lock):** Impede paralelismo real em threads Python, forçando execução serializada de código Python
2. **Cache misses:** Acesso não-sequencial à memória pode causar faltas de cache
3. **Operações de ponto flutuante:** Dominam o tempo de CPU em simulações numéricas

Gargalos de Sincronização

1. **Barreiras entre threads:** Todas as threads devem esperar pela mais lenta antes de prosseguir
2. **Locks para variáveis compartilhadas:** Contenção em escritas simultâneas ao erro máximo global

Gargalos de Comunicação (versão distribuída)

1. **Serialização:** Pickle é relativamente lento para arrays NumPy grandes
2. **Latência de rede:** Mesmo em localhost, há overhead significativo do protocolo TCP/IP
3. **Bandwidth:** Transmissão da malha completa a cada iteração consome largura de banda
4. **Ghost rows:** Necessidade de enviar linhas de vizinhança aumenta o volume de dados transmitidos

6. DESAFIOS ENFRENTADOS E SOLUÇÕES

6.1 Desafios Técnicos

Problema 1: Erros de Import no VSCode

Sintoma: `ModuleNotFoundError: No module named 'numpy'` ao executar o código no VSCode.

Causa: O VSCode estava utilizando um interpretador Python diferente daquele usado pelo pip para instalação de pacotes.

Solução adotada:

1. Criação de ambiente virtual isolado: `python -m venv venv`
2. Ativação do ambiente: `.\venv\Scripts\Activate.ps1` (Windows)
3. Instalação dos pacotes no ambiente virtual: `pip install numpy matplotlib pandas`
4. Configuração do interpretador correto no VSCode: `Ctrl+Shift+P` → Python: Select Interpreter

Problema 2: Worker Recebendo Sinal de Parada Prematuramente

Sintoma: Worker encerra após primeira iteração sem processar dados adequadamente.

Causa: As condições iniciais da matriz levavam a uma convergência imediata artificial, pois a diferença máxima entre iterações ficava abaixo da tolerância estabelecida.

Solução adotada:

1. Ajuste das condições iniciais com maior contraste térmico para forçar mais iterações
2. Aumento do tamanho da malha para forçar mais iterações antes da convergência
3. Redução da tolerância de convergência: $1 \times 10^{-4} \rightarrow 1 \times 10^{-6}$
4. Adição de logs de debug para monitorar `global_max_diff` e diagnosticar o comportamento

Problema 3: Performance da Versão Paralela Inferior ao Esperado

Sintoma: Speedup máximo de apenas $\approx 2,3\times$ com 8 threads (esperado: $\approx 6 - 7\times$ em cenário ideal).

Causa: O GIL do Python serializa a execução de código Python, impedindo paralelismo verdadeiro em operações CPU-bound.

Tentativas de solução avaliadas:

1. **Substituição por multiprocessing:** Eliminaría o GIL, mas adiciona overhead

de cópia de memória entre processos

2. **Vetorização com NumPy:** Reduz loops Python explícitos, mas ainda limitado por GIL em algumas operações

3. **Compilação com Numba:** Acelera loops críticos e libera GIL em código compilado

Solução parcial implementada:

- Manteve-se threading para simplicidade didática e clareza pedagógica
- Documentou-se limitação do GIL no relatório e código
- Propõe-se multiprocessing e Numba como trabalhos futuros

Problema 4: Overhead de Comunicação na Versão Distribuída

Sintoma: Speedup negativo para malhas pequenas ($\leq 200 \times 200$), com tempo de execução pior que a versão sequencial.

Causa: Tempo de serialização e transmissão via socket excede o ganho de paralelização computacional.

Análise quantitativa:

- Serialização de array 100×100 : $\approx 2-5$ ms
- Transmissão via localhost: $\approx 1-2$ ms
- **Total por iteração:** $\approx 6-10$ ms
- Tempo de computação por iteração (sequencial): $\approx 3-4$ ms

Conclusão: O overhead de comunicação (6-10 ms) supera o tempo de computação útil (3-4 ms), resultando em desempenho inferior.

Solução adotada:

1. Limitação dos testes a malhas $\geq 100 \times 100$ onde o problema é mais significativo
2. Uso de Pickle binário em vez de JSON para serialização mais rápida
3. Envio otimizado de apenas linhas necessárias (com ghost rows) em vez da malha completa

Melhorias propostas:

- Utilizar MPI (Message Passing Interface) via biblioteca mpi4py
- Implementar compressão de dados com algoritmos rápidos (lz4, zlib)
- Adotar comunicação assíncrona com buffers duplos para sobrepor comunicação e computação

6.2 Desafios de Desenvolvimento

1. Sincronização Complexa

Desafio: Garantir que todas as threads leiam os mesmos dados de uma iteração antes que qualquer thread atualize os dados.

Solução: Uso de `threading.Barrier` com dois pontos de sincronização críticos:

- Após computação (antes de trocar buffers de temperatura)
- Após troca de buffers (antes de iniciar próxima iteração)

2. Divisão de Trabalho Balanceada

Desafio: A última thread ou worker pode receber mais linhas se N não é divisível pelo número de threads/workers.

Solução: Distribuição equilibrada com tratamento especial para a última thread/worker.

3. Ghost Rows na Versão Distribuída

Desafio: Workers precisam de linhas vizinhas (ghost rows) para calcular derivadas espaciais nas bordas de seu domínio.

Solução: Master envia linhas adicionais (vizinhas) a cada worker, garantindo que todos tenham os dados necessários para os cálculos de fronteira.

7. PROPOSTAS DE MELHORIA

7.1 Melhorias de Curto Prazo

1. Substituir Threading por Multiprocessing

Justificativa: Elimina a limitação do GIL permitindo paralelismo verdadeiro.

Ganho esperado: Speedup próximo ao linear até o número de núcleos físicos disponíveis.

2. Vetorização Completa com NumPy

Justificativa: Operações vetorizadas do NumPy executam em código C nativo otimizado [23][26].

Ganho esperado: 10-50× mais rápido que versão sequencial com loops explícitos.

3. Utilizar MPI em vez de Sockets

Justificativa: MPI (Message Passing Interface) é otimizado especificamente para computação científica distribuída.

Ganho esperado: Redução de 50-70% no overhead de comunicação comparado a sockets TCP/IP.

7.2 Melhorias de Médio Prazo

4. Compilação JIT com Numba

Justificativa: Numba compila funções Python para código nativo e libera o GIL automaticamente.

Ganho esperado: 50-100× mais rápido + speedup linear com threads devido à liberação do GIL.

5. Implementação Híbrida MPI + OpenMP

Justificativa: Combina paralelismo distribuído (MPI) entre nós e paralelismo compartilhado (OpenMP/threads) dentro de cada nó.

Arquitetura:

- MPI para comunicação entre máquinas diferentes em um cluster
- OpenMP/threads para paralelização dentro de cada máquina

Ganho esperado: Escalabilidade eficiente para clusters com centenas de núcleos distribuídos.

7.3 Melhorias de Longo Prazo

6. Implementação em GPU com CUDA

Justificativa: GPUs possuem milhares de núcleos para computação paralela massiva [6][9].

Tecnologias disponíveis:

- CuPy (interface NumPy-like para CUDA)
- PyTorch/TensorFlow (frameworks de deep learning com suporte a GPU)
- Numba CUDA (JIT compilation para CUDA)
- PyCUDA (interface Python para CUDA)

Ganho esperado: 100-1000× para malhas grandes ($>1000 \times 1000$) aproveitando milhares de núcleos GPU.

7. Métodos Numéricos Avançados

Justificativa: Convergência mais rápida reduz drasticamente o número de iterações necessárias.

Técnicas propostas:

- **Multigrid methods:** Resolvem o problema em múltiplas escalas espaciais simultaneamente
- **ADI (Alternating Direction Implicit):** Esquemas implícitos mais estáveis que permitem passos de tempo maiores
- **Jacobi iterativo:** Método que paraleliza naturalmente
- **Red-Black Gauss-Seidel:** Permite paralelização eficiente através de coloração de malha

Ganho esperado: Redução de 50-90% no número de iterações necessárias para convergência.

8. CONCLUSÕES

8.1 Resultados Alcançados

Este trabalho atingiu com sucesso todos os objetivos propostos:

1. **Implementações funcionais:** Três versões completas e testadas do solver de difusão de calor 2D (sequencial, paralela e distribuída)
2. **Benchmarks sistemáticos:** Coleta de dados quantitativos de desempenho para múltiplos cenários computacionais
3. **Análise comparativa:** Identificação clara de trade-offs entre as diferentes abordagens
4. **Documentação completa:** Código bem comentado, repositório Git organizado e este relatório técnico detalhado

8.2 Principais Descobertas

1. **Threading em Python tem limitações severas:** O GIL impede speedup linear mesmo com código bem paralelizado, limitando os ganhos práticos. O speedup máximo observado foi de 2,34× com 8 threads para malha 100×100, muito abaixo do teórico.
2. **Overhead de comunicação é crítico:** A versão distribuída só compensa para problemas grandes devido ao alto overhead de serialização e transmissão via sockets. Para malhas pequenas (20×20), observou-se speedup negativo (0,76× com 4 workers).
3. **Lei de Amdahl é real:** A fração sequencial inevitável do código limita significativamente os ganhos práticos de paralelização. Mesmo com 95% do código paralelizável, o speedup máximo teórico é limitado a 20×.
4. **Escolha da ferramenta importa:** Python é excelente para prototipagem rápida e clareza pedagógica, mas apresenta limitações significativas para computação de alto desempenho em problemas CPU-bound.

8.3 Lições Aprendidas

Técnicas

- Paralelização eficiente de algoritmos iterativos usando decomposição de domínio
- Sincronização de processos concorrentes com barreiras e locks
- Implementação de comunicação em sistemas distribuídos via sockets TCP/IP
- Metodologia rigorosa de benchmarking e análise de performance

Conceitos

- Lei de Amdahl e suas implicações práticas para limitações teóricas de speedup
- Trade-offs fundamentais entre overhead de comunicação/sincronização e ganho computacional
- Importância crítica de escolher a linguagem e ferramentas adequadas ao tipo de problema
- Impacto do GIL na paralelização com threads em Python

Práticas

- Desenvolvimento colaborativo eficiente usando Git e GitHub
- Debugging sistemático de sistemas paralelos e distribuídos
- Documentação técnica de qualidade seguindo padrões acadêmicos
- Aplicação rigorosa de metodologia científica experimental em computação

8.4 Contribuições do Trabalho

Este trabalho contribui para a literatura de sistemas distribuídos ao:

1. Demonstrar empiricamente as limitações do GIL do Python em aplicações de computação científica paralela
2. Quantificar o overhead de comunicação via sockets em diferentes escalas de problema
3. Fornecer um caso de estudo completo e bem documentado de paralelização de EDPs
4. Propor um conjunto abrangente de melhorias baseadas em evidências experimentais

8.5 Trabalhos Futuros

Com base nos resultados e análises apresentados, sugerem-se as seguintes direções para trabalhos futuros:

1. Implementação da versão com multiprocessing e comparação detalhada com threading
2. Desenvolvimento de versão híbrida MPI + Numba para clusters
3. Implementação em GPU usando CUDA para avaliar ganhos em malhas muito grandes
4. Estudo comparativo de diferentes métodos numéricos (ADI, Multigrid) quanto a convergência e paralelização
5. Análise de escalabilidade em cluster real com múltiplas máquinas conectadas por rede

REFERÊNCIAS

AMRITKAR, A. R. **Parallel implementation and application of particle scale heat transfer in the discrete element method**. Virginia Tech, 2013. Disponível em: https://vtechworks.lib.vt.edu/bitstream/handle/10919/51192/Amritkar_AR_D_2013.pdf. Acesso em: 23 nov. 2025.

BUDEL, R. A. **Quantificação do risco na seleção e operação de recursos energéticos distribuídos inseridos em uma microrrede**. Dissertação (Mestrado em Engenharia Elétrica e Computação) — Universidade Estadual do Oeste do Paraná, Foz do Iguaçu, 2020.

DILGER, G. et al. **Descolonizar o imaginário: debates sobre pós-extrativismo e alternativas ao desenvolvimento**. São Paulo: Fundação Rosa Luxemburgo, 2016.

FRONTIERS IN COMPUTER SCIENCE. **Parallel computation to bidimensional heat equation using distributed systems**. 2024. Disponível em: <https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2023.1305800/full>. Acesso em: 23 nov. 2025.

GROPP, W.; LUSK, E.; SKJELLUM, A. **Using MPI: Portable Parallel Programming with the Message-Passing Interface**. 3. ed. Cambridge: MIT Press, 2014.

HEAT EQUATION MODEL. **2D Heat Equation Model - Python Implementation**. GitHub, 2020. Disponível em: <https://github.com/Ressnn/2DHeatEquationModel>. Acesso em: 23 nov. 2025.

LEVEQUE, R. J. **Finite Difference Methods for Ordinary and Partial Differential Equations**. Philadelphia: SIAM, 2007.

MATH LIBRETEXTS. **The Heat Equation**. 2024. Disponível em: [https://math.libretexts.org/Bookshelves/Differential_Equations/Introduction_to_Partial_Differential_Equations_\(Herman\)/10:_Numerical_Solutions_of_Partial_Differential_Equations/10.02:_The_Heat_Equation](https://math.libretexts.org/Bookshelves/Differential_Equations/Introduction_to_Partial_Differential_Equations_(Herman)/10:_Numerical_Solutions_of_Partial_Differential_Equations/10.02:_The_Heat_Equation). Acesso em: 23 nov. 2025.

NUMPY DEVELOPERS. **NumPy User Guide**. 2025. Disponível em: <https://numpy.org/doc/stable/user/>. Acesso em: 23 nov. 2025.

PACHECO, P. **An Introduction to Parallel Programming**. Burlington: Morgan Kaufmann Publishers, 2011.

PRESS, W. H. et al. **Numerical Recipes: The Art of Scientific Computing**. 3. ed. Cambridge: Cambridge University Press, 2007.

PYTHON SOFTWARE FOUNDATION. **socket — Low-level networking interface**. Python 3.11 documentation, 2025. Disponível em: <https://docs.python.org/3/library/socket.html>. Acesso em: 23 nov. 2025.

PYTHON SOFTWARE FOUNDATION. **threading — Thread-based parallelism**. Python 3.11 documentation, 2025. Disponível em: <https://docs.python.org/3/library/threading.html>. Acesso em: 23 nov. 2025.

RESEARCH COMPUTING SERVICES. **Limitations of Parallel Speedup**. GitHub Pages, 2020. Disponível em: <http://researchcomputingservices.github.io/parallel-computing/02-speedup-limitations/>. Acesso em: 23 nov. 2025.

SCIPY COMMUNITY. **The two-dimensional diffusion equation**. SciPython, 2025. Disponível em: <https://scipython.com/book2/chapter-7-matplotlib/examples/the-two-dimensional-diffusion-equation/>. Acesso em: 23 nov. 2025.

SILVA, M. F. et al. **Simulações de condução de calor unidimensional com o método de diferenças finitas**. Revista Brasileira de Ensino de Física, São Paulo, v. 45, 2023.

STRIKWERDA, J. C. **Finite Difference Schemes and Partial Differential Equations**. 2. ed. Philadelphia: SIAM, 2004.

THERMOPEDIA. **Parallel implementation**. 2012. Disponível em: <https://www.thermopedia.com/pt/content/9181/>. Acesso em: 23 nov. 2025.

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ. **Campus Cornélio Procopio**. 2022. Disponível em: <http://www.utfpr.edu.br/campus/cornelioprocopio>. Acesso em: 23 nov. 2025.

VISUALPDE. **Heat equation**. 2025. Disponível em: <https://visualpde.com/basic-pdes/heat-equation.html>. Acesso em: 23 nov. 2025.

WILKINSON, B.; ALLEN, M. **Parallel Programming: Techniques and Applications Using Networked Workstations and Parallel Computers**. 2. ed. Upper Saddle River: Pearson Education, 2004.

WIKIPEDIA. **Amdahl's law**. 2025. Disponível em: https://en.wikipedia.org/wiki/Amdahl's_law. Acesso em: 23 nov. 2025.

YOUTUBE. **Solving the Heat Diffusion Equation (1D PDE) in Python**. 2017. Disponível em: <https://www.youtube.com/watch?v=6-2Wzs0sXd8>. Acesso em: 23 nov. 2025.